

Bonus Exercise 1 Report

Algorithms for Uncertainty Quantification (IN2345), SS26

Aleksandra Topalova (03745233), Alexandra Savova (03731070), Julian Kraus (03734087)
Group 36

1 Assignment 1

1.1 Assignment 1.1

We computed the mean and variance for the same sample set using basic *Python* functionality and then *NumPy*'s functions for the same sample (See Table 1). We observed the following qualities of the quantities of interest:

- The **mean** remains the same in both using manual *Python* algorithms and *NumPy* methods, since the manual *Python* formula and *NumPy* both compute the average of the array elements by default.
- However, when computing the **variance**, when setting $ddof = 0$ in the *NumPy* method, the value differs because it computes population variance not sample variance, treating the data as the entire population rather than a subset. Setting $ddof = 1$ computes sample variance, using $N - 1$ as the divisor. More generally, $ddof$ stands for "Delta Degrees of Freedom" and is used in the divisor $N - ddof$.

DDOF	Method	Mean	Variance
0	Python	1.6471	0.1489
	NumPy	1.6471	0.1401
1	Python	1.6471	0.1489
	NumPy	1.6471	0.1489

Table 1: Comparison of basic *Python*- and *NumPy*-computed mean and variance values for the different DDOF values

1.2 Assignment 1.2

We computed the mean and covariance values of a multivariate random variable X (with 2 dimensions) for an increasing sample set size N , where $N \in \{10, 100, 1000, 10000\}$ with seeds 41, 42 and 43. We plotted the absolute error of the mean of the first dimension, first diagonal and upper right entry of the covariance matrix (Results in Figure 1). We also plotted the RMSE for both dimensions in Figure 2.

1.2.1 Absolute error observations

The lines in Figure 1 appear jagged, which comes from the sampling randomness, i.e. we can be "lucky" and get a sample set with mean and covariance that are close to the statistical ones (leading to a small absolute error) and vice versa. We sampled with 3 different seeds to get an overview of the general behavior. We observed that regardless of the spikes, the tendency is that the bigger the sample size, the smaller the error is. This can be explained the Law of Large Numbers: Increasing the number of samples reduces the variance (noise) of the result, which makes the average of the simulation closer to the statistically expected value.

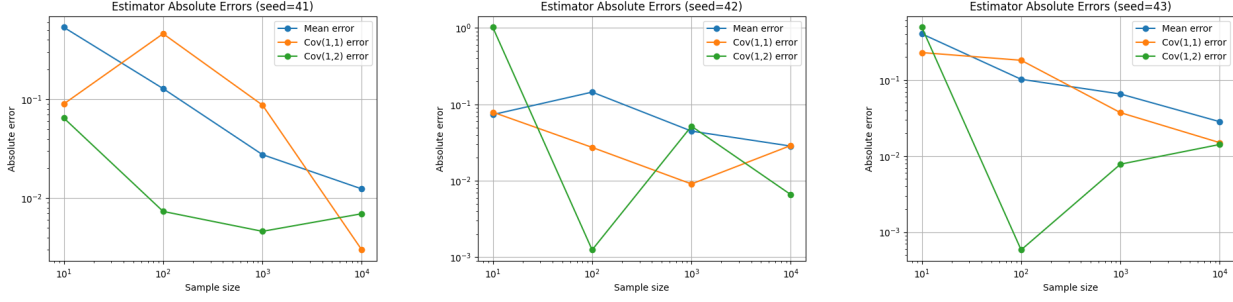


Figure 1: Absolute errors of mean and covariance for random seeds 41, 42 and 43

1.2.2 RMSE observations

Roughly averaged from all our multi-seed sampling, we get an RMSE value progression of $0.45 \rightarrow 0.14 \rightarrow 0.044 \rightarrow 0.014$. We see on Figure 2 that each value decreases with the expected factor of $N^{-1/2}$, making the graphs look close to a straight line. Here we don't see the fluctuations of the absolute error, because RMSE is the theoretical/statistical average size of the error. However, we can't use the formula for RMSE of a mean estimator $\hat{\sigma}_{\bar{X}_j} = \frac{\hat{\sigma}_{X_j}}{\sqrt{N}}$ for the covariance estimator. In the covariance estimator formula we have term multiplication, which represents the dependence of the multivariate random variable dimensions on one another in addition to the dependence on the mean. RMSE fails to capture this codependence.

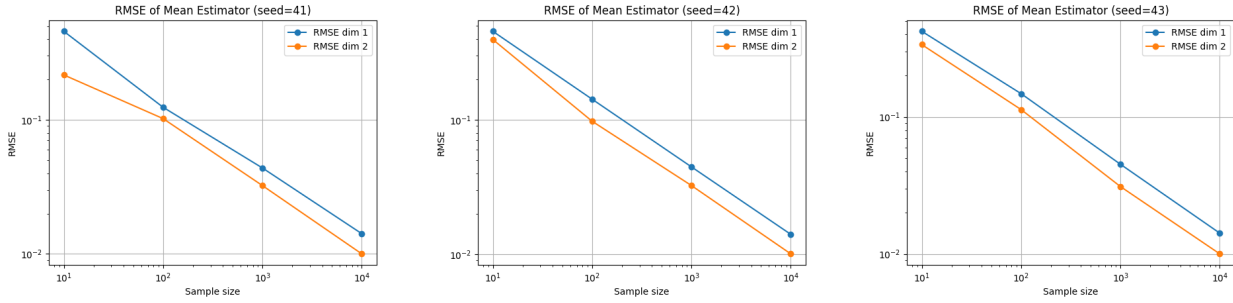


Figure 2: RMSE of both dimensions of X with seeds 41, 42 and 43

2 Assignment 2

2.1 Assignment 2.1

Similar to Assignment 1, on Figure 3 we see that the exact error fluctuates because of the randomness of the sampling of a batch, e.g. different spread. Hence, for 10 samples with $seed = 42$ we have an exact error closer to 10^{-2} , but for $seed = 43$ it is closer to 10^{-1} . RMSE again scales more predictably with roughly a factor of $N^{-1/2}$, making it appear as a straight line. Both values of interest decrease as we increase the amount of samples.

2.2 Assignment 2.2

Here we see the same type of discrepancy between exact error and RMSE as in Figure 3. The results from the transformed and non-transformed cases are identical, which is why the lines overlap, as the linear transformation does not negatively impact the uniformity of the distribution and both cases were sampled with the same seed. Additionally, comparing to Figure 3, the graphs have nearly identical shapes, however, in Figure 4 the results are scaled by a factor of $\alpha = b - a$, equal to 2 in this case.

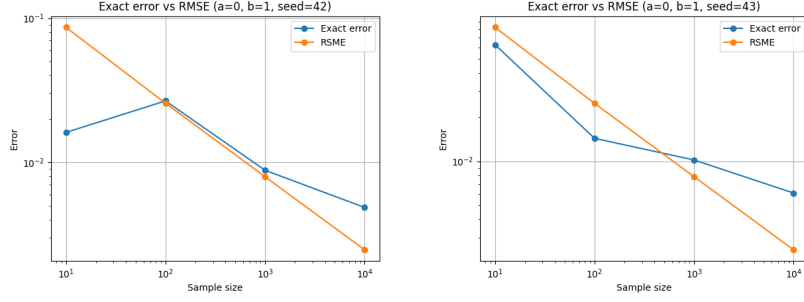


Figure 3: Exact error vs Monte Carlo RSME of integration of sin between 0 and 1

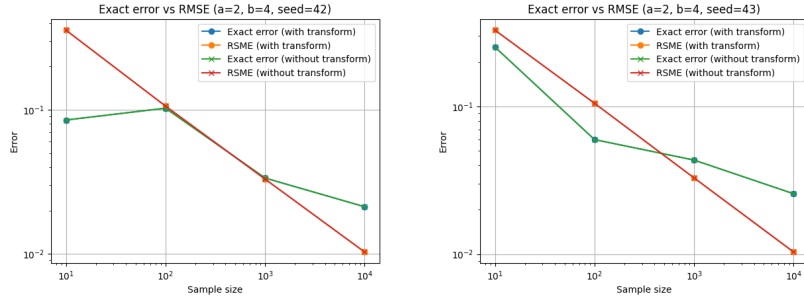


Figure 4: Exact error vs Monte Carlo RSME of integration of sin between 2 and 4. Distinction is made between with and without linear transformation

3 Assignment 3

N	Monte Carlo	CV h_1	CV h_2	CV h_3	IS Beta(5,1)	IS Beta(0.5,0.5)
10	0.038746	0.005176	0.005176	0.000907	0.526304	0.020013
100	0.046221	0.003523	0.003523	0.001228	0.223869	0.083207
1000	0.014795	0.001610	0.001610	0.000572	0.318954	0.032457
10000	0.010349	0.000500	0.000500	0.000124	0.210658	0.001706

Table 2: Computed values

3.1 Standard Monte Carlo

Errors decrease from 0.0387 at $N = 10$ to 0.0103 at $N = 10,000$, roughly consistent with the theoretical $O(1/\sqrt{N})$ rate. The non-monotone decrease (error rises slightly from $N = 10$ to $N = 100$) reflects the stochastic nature of the method.

3.2 Control Variates

All three control variates outperform standard MC. Notably, h_1 and h_2 produce identical errors at every N . This is expected: since $h_2(x) = 1 + h_1(x)$, adding a constant shifts neither the covariance with f nor the variance of ϕ , so the optimal coefficient $c^* = \text{Cov}(f, \phi) / \text{Var}(\phi)$ and the correction term $c^*(E[\phi] - \bar{\phi})$ are the same for both.

h_3 achieves the best results because $1 + x + x^2/2$ is the 2nd-order Taylor expansion of e^x , making it highly correlated with f and maximising variance reduction.

3.3 Importance Sampling

Both Beta proposals perform poorly compared to the control variate methods, and neither improves consistently over standard MC:

- **Beta(5,1):** PDF $\propto x^4$ concentrates samples near $x = 1$, assigning very small probability to $x \approx 0$. The resulting importance weights $p(x)/q(x) = 1/q(x)$ become enormous near zero, dramatically inflating variance. Errors are an order of magnitude larger than MC and fail to converge consistently.
- **Beta(0.5,0.5):** The arcsine distribution is U-shaped, concentrating at both endpoints. This is a poor match for the smoothly increasing e^x . The convergence is highly erratic: the error is lower than MC at $N = 10$ and $N = 10,000$, but worse at $N = 100$ and $N = 1,000$. This non-monotone behaviour is a consequence of the high variance of the importance weights: with a poorly matched proposal, small sample sizes are subject to large random fluctuations, preventing reliable convergence.

3.4 Findings

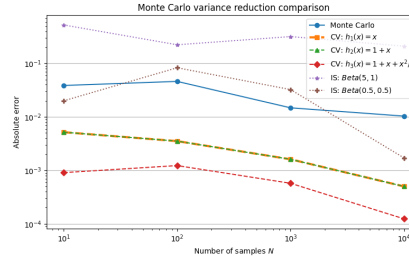


Figure 5: Absolute error comparison of Monte Carlo estimators for different sample sizes.

Figure 5 shows that the control variate estimators achieve the smallest errors overall, with $h_3(x) = 1 + x + x^2/2$ performing best due to its close approximation of e^x . Standard Monte Carlo improves more gradually as N increases, while the importance sampling estimators are less stable and generally produce larger errors.

4 Assignment 4

4.1 General Considerations

For this assignment, we utilized the provided linear damped oscillator model from *Tutor exercise 1*. Let the model be $G : X \rightarrow Y$, where X is the ODE and Y is the set of solutions (positions) of the oscillator for time points $t \in [0, 10]$ with $\Delta t = 0.01$. As a reminder, in the tutor exercise, the linear damped oscillator model was discretized and the value of $y(t)$ could be calculated by using either a custom-written Euler scheme or with the functionality of the *SciPy* library. The *SciPy* library was empirically faster for larger number of iterations due to optimizations. In this assignment, we calculate values for 1001 time points. Therefore, we selected the *odeint* method based on *SciPy* as our default ODE solver. When assuming no uncertainty for the parameters of X and, in particular, that ω is a known value, the approach is, as in the tutor exercise, to evaluate the ODE once for each required time-point of Y by using the "*odeint*" method and then plot the results.

4.2 Forward Propagation UQ

In this exercise, since $\omega \sim U(0.95, 1.05)$, we applied the four steps algorithm for forward propagation UQ from the exercise sheet as follows:

- Firstly, for the simulations, we modeled the deterministic parameters $x_d = \{c, k, f, y_0, y_1\}$ of X , as well as its stochastic parameters $x_r = \{\omega\}$ in the code of *assignment_4.py*.

- Secondly, for each of the required sample sizes $n \in N = [10, 100, 1000, 10000]$, we sampled n i.i.d. values for ω in two different ways - with Standard Monte Carlo sampling and with Quasi-Monte Carlo based on Halton sequences approach. For the sampling itself, we used the functionality of the *Chaospy* library.
- Thirdly, we propagated the uncertainty of ω through G . For each of $n \in N$ and the respective n samples of ω , we evaluated G or in other words, simulated the oscillator. For clarity of the code solution, let us give an overview of the resulting data structures. They are two matrices - one for the Standard Monte Carlo sampling and one for the Quasi-Monte Carlo based on Halton sequences. Both matrices contain $|N| = 4$ rows, i.e. a dedicated row for each of the different number of samples $n \in N$. Each of the rows contains a $n \times |T|$ matrix where T is the "set of time points" and $|T| = 1001$. The $n \times 1001$ matrices themselves contain the sample oscillator simulation results over the 1001 time points for the n sampled values of ω .
- The fourth and last step of the algorithm was to estimate the quantities of interest, mean and variance, for $t = 10$ for each approach (Monte Carlo sampling and Quasi-Monte Carlo based on Halton sequences) and for each $n \in N$. We extracted the values of $y(10)$ from the total sampling results and used *NumPy* for the statistical operations. Notably, we calculated the sample variance by setting $ddof = 1$, like in Assignment 1). Then, to analyze the results, we calculated the relative error to the provided reference values for mean and variance of $y(10)$. We chose to code the solution of the assignment in such a way that a different t than 10 can easily be selected for plotting and statistics calculations, in case the reference quantities of interest for Y are not available for $t = 10$. Lastly, we plotted the results as requested in the exercise sheet. We will discuss the plots below.

In terms of data structures in the code, our preferred approach was to predefine empty *NumPy* arrays to allow for memory optimizations. In cases where the data structures were not uniformly sized or the data indexing via keys was beneficial for code readability, we opted for maps (dictionaries).

4.3 Findings

The assignment called for the plotting of three graphs, the first two of which can be seen in Fig. 6.

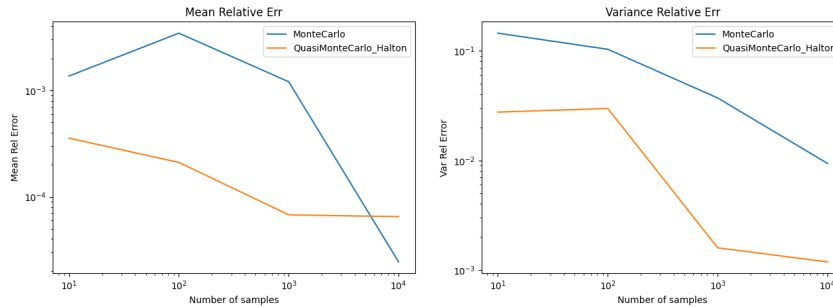


Figure 6: Relative errors of mean and variance at $t=10$ for different sample sizes

The left side graph of Fig. 6 visualizes the relative error of the mean for $y(10)$ and $n \in N$ for both Monte Carlo sampling and Quasi Monte Carlo based on Halton sequences. The right side graph visualizes the relative error of the variance for the same parameters. For all cases on both graphs, with the highest numbers of samples, the error becomes smaller. Thus, the expectation from the lecture (Lecture 4, slide 5 "Towards More Advanced Sampling Techniques") that increasing N is beneficial to the increased accuracy of standard and advanced Monte Carlo sampling, is also evident for the case of forward uncertainty quantification propagation within the linear damped oscillator model. Nonetheless, between $n = 10$ and $n = 100$, we see a slight increase in the mean relative error for Monte Carlo sampling. This shows that in the case of forward propagation of UQ and calculation of relative error instead of standard error, the relationship between convergence of the error and N is more complicated than $O(\sqrt{N})$ (as is in the case of no propagation and standard error for Monte Carlo). Had the convergence rate of the relative error been $O(\sqrt{N})$, the lines would have been straight on the logarithmic scale. In addition, it can be seen on the graphs that Quasi Monte Carlo based on Halton sequences has increased accuracy even on smaller n , due to its properties that decrease σ_f^2 , as it was explained in the lecture.

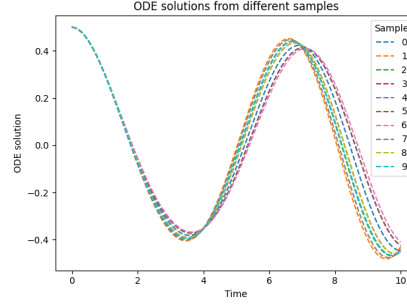


Figure 7: Ten solution trajectories, standard Monte-Carlo

Fig. 7 visualizes the oscillation trajectories of 10 solutions of the ODE plotted over time $t \in [0, 10]$. The 10 solutions result from 10 different standard Monte-Carlo samples of ω . On this specific graph, we depict the 10 solutions calculated for the 10 ω samples for the $n = 10$ part of the assignment. While there is a certain visible variance of the oscillator positions between the ten solutions, the linear damped oscillator model seems to be not too sensitive to the uncertainty of ω , making it a suitable model for uncertainty quantification approaches. In the same line of thought, the linear damped oscillator as a problem is well-posed and suitable for computer-based solution.

In the code for the assignment, we used a parameter configuration of the simulations, so that it is easy to generate and compare instead a visualization of the 100 trajectories for the 100 Monte Carlo samples of ω for $n = 100$, as shown in Fig.8. This figure shows that the higher number of samples does not lead to higher visual variance in the ODE solutions, again confirming the good properties of the model and the problem.

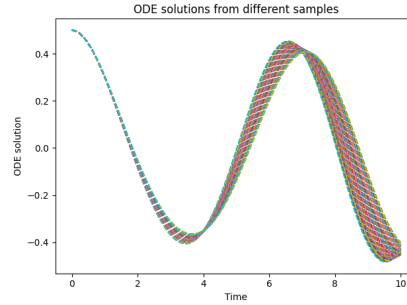


Figure 8: 100 solution trajectories, standard Monte-Carlo